

Step 2 — Style Reviewer With RAG — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from langchain_community.document_loaders import TextLoader
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel

docs = TextLoader("style_guide.md").load()
vectorstore = Chroma.from_documents(docs, OpenAIEmbeddings())
retriever = vectorstore.as_retriever()

class Finding(BaseModel):
    severity: str
    message: str

class StyleReview(BaseModel):
    findings: list

def run_style_review(diff):
    chunks = retriever.invoke(diff)
    guide_text = "\n".join(c.page_content for c in chunks)
    prompt = f"Style guide:\n{guide_text}\n\nCheck this diff against the style gu
    model = ChatOpenAI(model='gpt-4o-mini').with_structured_output(StyleReview)
    return model.invoke(prompt)
```

This works — the style guide gets chunked, embedded, and retrieved chunks get inserted into the prompt before the model sees the diff. Loading the whole file as one `TextLoader` document means Chroma is splitting on nothing (one giant chunk), which defeats most of the point of retrieval on anything longer than a page or two.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — real chunking, typed output, Security turned into a tool-using agent

`agents/style_reviewer.py`

```
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_chroma import Chroma
from pydantic import BaseModel, Field

class Finding(BaseModel):
    severity: str = Field(description="One of: critical, warning, info")
    message: str = Field(description="The specific style-guide rule violated, and")

class StyleReview(BaseModel):
    findings: list[Finding]

def build_style_retriever(guide_path: str = "style_guide.md"):
    docs = TextLoader(guide_path).load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = splitter.split_documents(docs)
    vectorstore = Chroma.from_documents(chunks, OpenAIEmbeddings())
    return vectorstore.as_retriever(search_kwargs={"k": 4})

def run_style_review(retriever, diff: str) -> StyleReview:
    retrieved = retriever.invoke(diff)
    guide_text = "\n\n".join(chunk.page_content for chunk in retrieved)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output
    return model.invoke(
        f"You are a style reviewer. Only flag violations of the style guide excerpt"
        f"below – do not use general style conventions that aren't stated here. "
        f"If nothing in these excerpts is violated, return an empty findings list"
        f"Style guide excerpts:\n{guide_text}\n\nDiff:\n{diff}"
    )
```

agents/security_agent.py — turning Step 1's plain chain into an agent with a real tool

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
import re

SECRET_PATTERNS = [
    re.compile(r'(?i)(api[_-]?key|secret|password|token)\s*=\s*["\'](?:[^\s\']{8,}|"\s*\s*["\'])'),
    re.compile(r'sk-[a-zA-Z0-9]{20,}'),
]

@tool
def scan_for_secret_patterns(added_lines: str) -> list[str]:
    """Scan added diff lines for hardcoded secret/API key patterns using regex.
    Always call this before concluding a diff has no secrets."""
    matches = []
    for pattern in SECRET_PATTERNS:
        matches.extend(m.group(0) for m in pattern.finditer(added_lines))
    return matches

def run_security_review(topic_diff: str):
    agent = create_react_agent(ChatOpenAI(model="gpt-4o-mini"), tools=[scan_for_s
    return agent.invoke({"messages": [("user", f"Security review this diff, using
```

Difference from Basic: `RecursiveCharacterTextSplitter` actually splits the guide into retrievable chunks instead of one giant blob, `k=4` limits how many chunks come back, and the prompt explicitly instructs the model to only flag what's in the retrieved excerpts and to say nothing if nothing's violated — closing off the "invents a plausible-sounding rule" failure mode a vaguer prompt invites. Security becomes genuinely agent-shaped: it now decides to call a real tool (a regex secret scanner, matching Step 1's Advanced hint) rather than just producing structured output from one direct prompt. Neither piece yet *proves* the Style reviewer's answer is actually grounded in the retrieved text, or that the Security agent actually called its tool — that's what Advanced verifies.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a `quote` field per finding, verified against the retrieved chunks

```
class Finding(BaseModel):
    severity: str = Field(description="One of: critical, warning, info")
    message: str = Field(description="The specific style-guide rule violated, and")
    quote: str = Field(description="The exact sentence from the style guide excerpt")

class StyleReview(BaseModel):
    findings: list[Finding]

class UngroundedFindingError(Exception):
    pass

def run_style_review(retriever, diff: str) -> StyleReview:
    retrieved = retriever.invoke(diff)
    guide_text = "\n\n".join(chunk.page_content for chunk in retrieved)

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(StyleReview)
    review = model.invoke(
        f"You are a style reviewer. Only flag violations of the style guide excerpt below. For every finding, include the exact quote from the excerpts that supports it, word for word. If nothing is violated, return an empty list."
        f"Style guide excerpts:\n{guide_text}\n\nDiff:\n{diff}"
    )

    for finding in review.findings:
        if finding.quote not in guide_text:
            raise UngroundedFindingError(
                f"Finding '{finding.message}' cites a quote not found in the retrieved style guide excerpts – likely answered from general knowledge, not the retrieved excerpts."
            )

    return review
```

Expected behavior: a genuine violation of a rule actually in your style guide passes through with its quote verified. If the model ever invents a plausible-sounding rule that isn't in the retrieved text (the exact failure this project's problems table warns about), the quote check catches it and raises `UngroundedFindingError` instead of silently returning an ungrounded finding as if it were real.

Approach 2 — verifying the Security agent actually called its tool

```
class ToolNotCalledError(Exception):
    pass

def run_security_review(diff_text: str):
    agent = create_react_agent(ChatOpenAI(model="gpt-4o-mini"), tools=[scan_for_s
    result = agent.invoke(
        {"messages": [{"user", f"Security review this diff, using the scan_for_se
        {"recursion_limit": 8},
    )

    tool_was_called = any(
        getattr(message, "name", None) == "scan_for_secret_patterns"
        for message in result["messages"]
    )
    if not tool_was_called:
        raise ToolNotCalledError("Security agent answered without running the sec

    return result
```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate trusts that "only use the retrieved excerpts" and "always call the scan tool first" hold just because the prompt asked for them — exactly the same trust-without-verifying gap Project 4's Step 3 and Step 4 Advanced hints both closed for Research and Analysis. Approach 1 closes it for the Style reviewer, by forcing every finding to cite a real, checkable quote from what was actually retrieved. Approach 2 closes it for the Security agent, by confirming — the same way Project 4's Research agent verification worked — that the tool call genuinely happened, not just that a plausible answer came back.

Which one should you actually write? Both — they're the same underlying idea (verify grounding/tool use actually happened, don't just prompt for it and hope) applied to this project's two different reviewers. This is also the real generalization test the project's own charter names: if you recognize "this is the same verification pattern from Project 4, just applied to RAG instead of a research tool," that's the proof the pattern actually transferred, not just the code.